Spring Boot 入門

JAC0177G-L1-04

~サーブレット&JSP からのステップアップ~

演習ガイド

商標

- アマゾン ウェブ サービス、AWS および Amazon Web Services ロゴは、米国その他の諸国における、Amazon.com, Inc.またはその関連会社の商標です。
- Cisco、Cisco Systems、Cisco Systems ロゴは、Cisco Systems, Inc.および関連会社の米国ならびに他の国における商標もしくは登録商標です。
- IIBA®、BABOK® は、IIBA® (International Institute of Business Analysis™) の登録商標です。
- Intel、インテルは、米国およびその他の国における Intel Corporation の商標です。
- ITIL®は、PeopleCert group の登録商標であり、PeopleCert Group の許可のもとに使用されています。すべての権利は留保されています。
- Microsoft、Windows は、米国 Microsoft Corporation の、米国およびその他の国における登録商標または商標です。
- Oracle と Java は、Oracle Corporation およびその子会社、関連会社の米国およびその他の国における登録商標です。
- PMI、PMP、PMBOK は、プロジェクトマネジメント協会(Project Management Institute, Inc.)の登録商標です。
- VMware、VMware ロゴは、VMware, Inc.の米国および各国での商標または登録商標です。
- BOOT CAMP、NEW TRAIN、MIX TRAIN、TRAINOCAMP は、トレノケート株式会社の登録商標です。

その他の会社名、製品名およびサービス名は、それぞれ各社の商標または登録商標です。

目　次

演習 1 Spring Boot と開発環境の概要 7

演習 1.1. フレームワークとは 9

演習 1.2. Spring Framework 9

演習 1.3. Spring Boot とは 10

演習 1.4. Maven でライブラリ管理 10

演習 2 Spring Web アプリケーション 11

演習 2.1. 入力画面へ遷移するメソッド作成 13

演習 2.2. BMI 計算用のメソッド作成 14

演習 2.3. 計算済みの BMI 数値を結果ページへ表示 15

演習 2.4.【オプション】 Form クラスを利用 16

演習 3 Spring Data JPA 17

演習 3.1. Entity と Repository の定義 20

演習 3.2. 全社員一覧機能の実装 21

演習 3.3. 社員番号検索機能の実装 23

演習 3.4. 給与による社員検索機能の実装 24

演習 3.5. 社員データ登録機能の実装 26

演習 3.6.【オプション】名前検索機能の実装 26

演習 3.7.【オプション】 削除機能の実装 28

演習 3.8.【オプション】Service クラスの利用 28

演習 4 セッションと入力チェック 29

演習 4.1. ID とパスワードからのログイン機能の実装 31

演習 4.2. セッションデータの取得と利用 33

演習 4.3. 入力チェック機能の実装 34

演習 4.4.【オプション】登録画面に入力チェック機能の実装 36

演習 5 Thymeleaf と Web デザイン 37

演習 5.1. CSS と JavaScript の適用 39

演習 5.2. 共通レイアウトを作成し、全画面に適用 41

演習 5.3. イメージの動的挿入 44

演習 5.4.【オプション】権限による給与データの動的表示 45

演習 5.5.【オプション】顔写真の全画面適用 46

演習 5.6.【オプション】権限による「給与検索」の動的表示 46

演習 6 外部参照と複雑な SQL 操作 47

演習 6.1. エンティティ・リポジトリの追加と修正 51

演習 6.2. 外部参照を持つデータの全件検索 53

演習 6.3. 社員番号を使った個別検索 54

演習 6.4.【オプション】存在しない社員番号を考慮する個別検索 55

演習 6.5. 社員情報の登録 57

演習 6.6. JPQL を使った検索(給与+勤務地) 60

演習 7 Bootstrap 63

演習 7.1. Bootstrap 導入前後の変化を確認 67

演習 7.2. Bootstrap の基本スタイルの適用 68

演習 7.3. Bootstrap のテーブルデザインの適用 70

演習 7.4. グリッドシステムで画面レイアウトを整える 72

演習 7.5. フォームの改善 74

演習 7.6.【オプション】共通レイアウト画面に変更 76

1

演習 1　Spring Boot と開発環境の概要

演習の目標

- Spring Framework の特徴について理解できる
- Spring Boot の長所について理解できる
- Maven でライブラリを管理できる

演習の概要

第 1 章で登場した重要単語の意味を確認します。

予想所要時間

5 分

演習1.1. フレームワークとは

問題 1:フレームワークを使用する利点は何ですか？

a) システム開発の時間と労力が増える。

b) 特定の種類のシステムをより速く、効率的に作ることができる。

c) オブジェクト指向プログラミングを学ぶ必要がない。

d) エラーメッセージが一切表示されなくなる。

問題 2:フレームワークを使う際の注意点はどれですか？

a) フレームワークは自由度が高く、ルールに従う必要がない。

b) エラーメッセージは無視しても良い。

c) フレームワークに定められたルールや方法に従う必要がある。

d) データベースに接続する際はフレームワークを使わない方が良い。

演習1.2. Spring Framework

問題 1: Spring Framework の特徴はどれですか？

a) DI(依存性注入)を使用しない。

b) MVC(Model-View-Controller)モデルをサポートしない。

c) 他のフレームワークやライブラリと組み合わせて利用することが可能。

d) Struts2 や、Seasar2 などのフレームワークと競合している。

問題 2: Spring Framework における DI(依存性注入)の主な目的は何ですか？

a) オブジェクト間の依存関係をハードコーディングする。

b) よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める。

c) Java の基本的な構文を変更する。

d) Web ページのデザインとレイアウトを管理する。

演習1.3. Spring Bootとは

問題 1:Spring Bootによる初期環境構築のメリットは何でしょうか？

a) 設定が全く必要ない。

b) 設定を自動化して初期環境構築を簡単にする。

c) 設定がより複雑で細かい。

d) 開発者がすべての設定を手動で行う必要がある。

問題 2: Spring Bootが提供するBeanに関する機能として正しいものはどれですか？

a) 必要なBeanをすべて手動で登録する必要がある。

b) 開発者が用意したクラスを自動でBeanとして登録してくれる。

c) Beanの登録機能は提供されていない。

d) Beanを登録するにはXMLファイルが必須である。

演習1.4. Mavenでライブラリ管理

問題 1: Mavenを使用する主な理由は何ですか？

a) プロジェクトのデザインを改善するため。

b) ソースコードの書き方を自動化するため。

c) ライブラリのダウンロードと参照関係の管理を自動化するため。

d) データベース接続を自動で設定するため。

問題 2: Mavenを使用して、どのようにライブラリを管理すればいいですか？

a) ライブラリを手動でダウンロードしてプロジェクトに追加する。

b) ライブラリの情報をpom.xmlファイルに記述して、Maven installを実行する。

c) すべてのライブラリを個別にインストールする。

d) ライブラリの情報をプロジェクトのREADMEファイルに記述する。

2

演習 2　Spring Web アプリケーション

演習の目標

- Web アプリケーションの基本的な流れを理解できる
- Controller を作成し、利用できる
- View から Controller へのリクエストパラメータを取得できる
- Form クラスを使用してリクエストパラメータを取得できる
- Model を通じて Controller から View にデータを表示できる

演習の概要

第 2 章で学習した Spring アプリケーションの基本文法を確認し、実際にプログラムを作成します。

- 身長と体重を入力する画面作成 BMI 計算のツールを作成
- 入力されたデータを基に、BMI 計算を行う Controller を作成
- 計算結果を表示する画面を作成

予想所要時間

演習 2.1　10 分

演習 2.2　25 分

演習 2.3　15 分

演習2.1. 入力画面へ遷移するメソッド作成



a. BMIController に、適切なアノテーションを付けてください。

- テキスト 2.2 の【 作業手順①】を参考にしてください。

b. BMIController に、入力画面(index.html)へ遷移するメソッドを作成してください。

- テキスト 2.2 の【 Index メソッドの解説】を参考にしてください。
- index.html は、作成済みです。ソースコードを確認してください。
- Get マッピングする URL は、“/” にしてください。

  ※return には、「 .html」を省略可能です。

演習2.2. BMI 計算用のメソッド作成

a. 計算用のメソッドに、Post 形式のマッピングをしてください。

- テキスト 2.3 の【 作業手順③】を参考にしてください。
- マッピングする URL は、“/bmi”にしてください。

b. index.html の name 属性を参考にして、身長と体重のパラメータを取得するように引数を設定してください。(Form クラス未使用)

- テキスト 2.3 の【 作業手順③】を参考にしてください。
- 小数点でも受け取れるように、型は double にしてください。

c. 取得した身長と体重を利用し、bmi 計算を行ってください。

- テキストは参照せずに、以下の手順通りに、コーディングをしましょう。
- 身長はメートルの単位に変換(身長÷100)して、別途の変数に代入してください。
- bmi 計算を行い、double bmi 変数に代入してください。
  ※BMI 計算式 ： 体重÷(メートル変換済みの身長 × メートル変換済みの身長)

演習2.3. 計算済みの BMI 数値を結果ページへ表示



a. 計算用メソッドの引数に、Model model を追加してください。

- テキスト 2.5 の【 作業手順③】を参考にしてください。

  Model は、インポートも忘れないで下さい。(import org.springframework.ui.Model;)

b. bmi 結果の変数を、model の中に格納してください。

- テキスト 2.5 の【 作業手順③】を参考にしてください。
- model.addAttribute を使ってください。キー名は、”bmi”にしてください。

c. result.html へ遷移させてください。

d. result.html に あなたの BMI は ～ です」と表示するように実装してください。

- テキスト 2.5 の【 作業手順④】を参考にしてください。
- p タグの中に、th:text 属性を追加してください。

  ※ th:text = “${ model に格納されている**キー名** } ”

  ※ form データを受け取るわけではありませんので、「 .」で繋げず bmi 単体を指定します。

■ **動作確認：http://localhost:8080/**

- 身長と体重を入力し、正しく BMI 値が出力されるかを確認

演習2.4.【 オプション】 Form クラスを利用

Form クラスを利用するように、修正してください。

- テキスト 2.4 を参考にしてください。

a. form パッケージの BMIForm クラスに追記してください。

- double 型で、身長(height)、体重(weight)、bmi のフィールドを追加してください。
- Getter & Setter を追加してください。

b. BMIController に戻り、メソッドの引数に BMIForm を加えるように、修正してください。

c. get メソッドを使い、値を計算してください。計算後は、setBmi メソッドで bmi をセットしてください。

d. model に bmiForm を格納するよう、修正してください。

e. result2.html へ遷移するよう、修正してください。

f. result2.html の方で、model 格納した Form データから bmi を表示してください。

- **動作確認：http://localhost:8080/**
  - 身長と体重を入力し、正しく BMI 値が出力されるかを確認

3

演習 3　Spring Data JPA

演習の目標

- Spring Data JPA の基本概念を理解できる
- Controller を作成し、利用できる
- エンティティとレポジトリーを実装できる
- 全件検索・条件検索ができる
- 登録・更新・削除ができる(オプション)

演習の概要

第 3 章で学習した Spring Data JPA の基本文法を確認し、実際にデータベースと連動する人事管理プログラムを作成します。

- 全社員検索機能を持つメソッドと、「 全社員一覧」画面の作成
- 社員番号で社員を検索する機能を持つメソッドと、画面の作成
- 給与データで社員を検索する機能を持つメソッドと、画面の作成
- 社員データを登録する機能を持つメソッドと、画面の作成

予想所要時間

演習 3.1　10 分

演習 3.2　20 分

演習 3.3　15 分

演習 3.4　10 分

演習 3.5　25 分

■データベース情報

※データベースは、既に構築済みです。

src/resources/application.properties を確認してください。

＜接続情報＞

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

＜employee テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	−	−	−
3	社員名	name	VARCHAR	−	−	−
4	給与	salary	INT	−	−	

<Employee テーブルの初期データー>

id	password	name	salary
1001	password	山田太郎	230000
1002	password	鈴木一郎	230000
1003	password	田中花子	300000
1004	password	山田次郎	400000
1005	password	高橋美智子	500000
1006	password	平恵	270000
1007	password	佐々木舞	330000
1008	password	中村翔	350000

演習3.1. Entity と Repository の定義

a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト 3.2 の【 作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト 3.2 の【 作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

c. repository.EmpRepository に JpaRepository を適切に継承してください。

- テキスト 3.3 の【 作業手順④】を参考にしてください。
- 〈紐づけたい Entity 名, Entity の主キーの型名〉

演習3.2. 全社員一覧機能の実装



a. この演習では、初期画面として「社員検索」画面を利用します。
この画面から、それぞれの Controller メソッドを呼び出します。

- src/main/resources/templates/index.html のソースコードを確認してください。

b. データベースからデータを取得したいため、Employee テーブルを操作できる EmpRepository の「依存性の注入」を行ってください。

- EmpController に戻り、EmpRepository の依存性の注入を行ってください。
- テキスト 3.3.1 を参考にしてください。

c. EmpRepository を利用して、全件検索を行ってください。

- テキスト 3.4 の【作業手順①】を参考にしてください。
- b.で指定した EmpRepository の変数を利用して、findAll メソッドを呼び出してください。
- 複数の行を取得するため、戻り値型は List<Employee>です。

d. 取得したリストを HTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「“empList”」にしてください。
- 遷移先は、「emp_list.html」です。

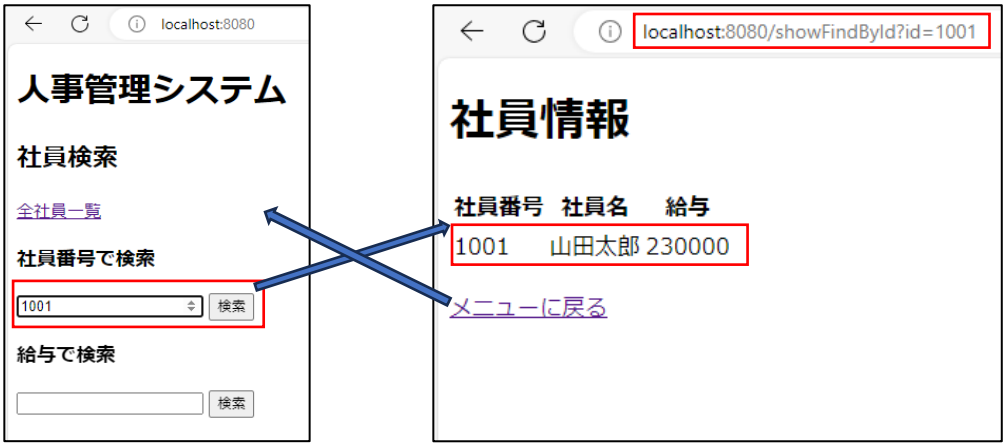
e. emp_list.html に model 内のデータを出力してください。

- テキスト 3.4 の【 作業手順②】を参考にしてください。
- パスワード以外の列を全て出力してください。
- 「 :」を境に右は Model のキー名の「 empList」、左は Model の中に入っているものを意味する任意の変数名にしてください。(emp など)

■ **動作確認：http://localhost:8080/**

- 「 全社員一覧」リンクをクリックし、社員リストが表示されるか確認してください。

演習3.3. 社員番号検索機能の実装



a. src/main/resources/templates/index.html の「 社員番号で検索」のソースコードを確認してください。

- テキスト 3.5 の【 作業手順①～②】が該当する内容です。
- input type = number は、整数のみ入力できるテキストフォームを生成します。
- name 属性の値を覚えてください。

b. EmpController クラスの showFindById メソッドの引数に、a.の name 属性値を追記してください。

- テキスト 3.5 の【 作業手順③】を参考にしてください。

c. EmpRepository を利用して、主キー検索を行ってください。

- テキスト 3.5 の【 作業手順③】を参考にしてください。
- EmpRepository の変数を利用して、findById メソッドを呼び出してください。

  ※.orElse(null)は必ず付けてください。
- 1 行のデータを取得するため、戻り値型はエンティティ名の Employee です。

d. 取得したリストを HTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「 “emp”」にしてください。
- 遷移先は、「 emp.html」です。

e. emp.html に model 内のデータを出力してください。

- パスワード以外の列を全て出力してください。
- テキスト 3.5 の【 作業手順④】を参考にしてください。

■ **動作確認：http://localhost:8080/**

- 「 社員番号で検索」フォームに「 1001」と入力し、[検索]ボタンをクリックすると、正しく社員情報ページが表示されるか確認してください。

演習3.4.　給与による社員検索機能の実装



a. src/main/resources/templates/index.html の「 給与で検索」のソースコードを確認してください。

- テキスト 3.5.1 の【 作業手順①】の内容が該当します。
- name 属性の値を覚えてください。

b. EmpRepository に salary 列検索用のメソッドを定義してください。

- テキスト 3.5.1 の【 作業手順②】を参考にしてください。
- 複数行の検索結果を取得する可能性があるため、戻り値型を List<Employee>にします。

c. EmpController クラスの showFindBySalary メソッドの引数に、a.の name 属性値を追記してください。

- テキスト 3.5.1 の【 作業手順③】を参考にしてください。

d. EmpRepository を利用して、主キーではない列(salary)検索を行ってください。

- テキスト 3.5.1 の【 作業手順③】を参考にしてください。
- b.で定義したメソッドを呼び出してください。
- 複数の行を取得するため、戻り値型は List<Employee>です。

e. 取得したリストを HTML に出力させるため、model にリストを格納してください。

- Model のキー名は、「“empList”」にしてください。
- 遷移先は、「emp_list.html」です。
- emp_list.html は、演習 3.2 で定義済みですので、修正事項はありません。

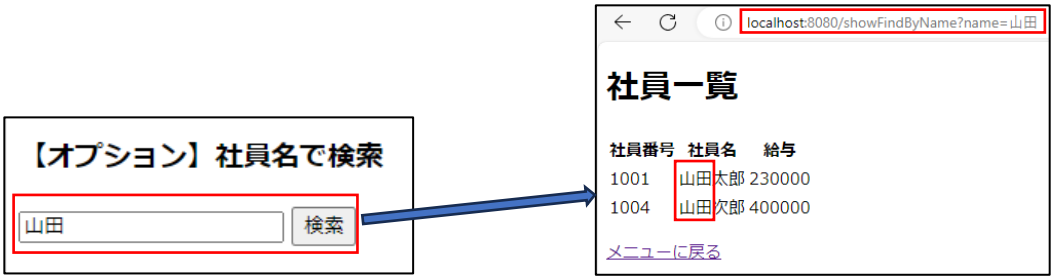
■ **動作確認：http://localhost:8080/**

- 「給与で検索」フォームに「230000」と入力し、[検索]ボタンをクリックすると、2 件の社員リストが表示されるかを確認してください。

演習3.5.【 オプション】 名前検索機能の実装

時間内に 3.4 まで完了できた方は、オプション問題に挑戦してください。

細かい修正箇所の案内はありませんので、学習した内容を基に実装してください。



a. EmpRepository に、新たに name の Like 検索を行うメソッドを定義してください。

- テキスト 3.5.3 を参考にしてください。
- 複数の行を取得するため、戻り値型は List<Employee>です。
- emp_list.html へ遷移させてください。

演習3.6. 社員データ登録機能の実装



a. EmpController の showInputView メソッドのソースコードを確認してください。

b. src/main/resources/templates/emp_input.html のソースコードを確認してください。

- テキスト 3.6 の【 作業手順②】の内容が該当します。
- action 属性値が[“insertEmp”]、method 属性値が[“post”]だということを覚えてください。

- 3 つのテキストフォームがあり、それぞれ name、password、salary の name 属性値が入っていることを覚えてください。

c. form.EmpForm.java を確認してください。

- テキスト 3.6 の【 作業手順③】の内容が該当します。

d. EmpController に戻り、メソッドの引数に、EmpForm と Model を入れてください。

- テキスト 3.6 の【 作業手順④】を参考にしてください。

e. Employee のエンティティを new で生成し、
Form からのデータを空っぽの Entity にコピーしてください。

- テキスト 3.6 の【 作業手順④】を参考にしてください。

f. EmpRepository の変数を利用して、save メソッドを呼び出してください。

- 引数には、データのコピー済みのエンティティを入れます。
- 登録処理の後は、戻り値を model に格納し、登録データを確認するために、result.html へ遷移させます。

■ **動作確認：http://localhost:8080/**

- [社員データ登録]リンクをクリック
- 登録する社員データを入力し、[登録]ボタンをクリック
- 入力したデータ通りに、新規データが登録されているかを確認

演習3.7.【 オプション】 削除機能の実装



社員情報ページの削除ボタンをクリックすると、その社員のデータを削除する機能を実装してください。

a. emp.html の削除ボタンのコメントを外してください。

b. delete 処理をするメソッドを実装してください。

- テキスト 3.8 を参考にしてください。
- 削除処理後、全件検索の URL へリダイレクトしてください。

演習3.8.【 オプション】Service クラスの利用

テキスト 3.9 を参考にして、今回実装したメソッドのビジネスロジックを Service クラスに分離する実装に変えてください。

URL は、localhost:8080/emp2/ で実装してください。

4

演習 4　セッションと入力チェック

演習の目標

- ID とパスワードによる認証処理を実装できる
- セッションへのデータ保持と、活用方法について理解できる
- 入力チェックとエラーハンドリングを理解し、実装できる

演習の概要

第 4 章で学習したセッションと入力チェックの基本文法を確認し、実際にデータベースと連動するログインプログラムを作成します。

- ID とパスワードの整合性をチェックする機能を持つメソッドの作成
- ログイン成功時、ユーザ情報をセッションに格納する機能を持つメソッドの作成
- 入力値の整合性をチェックする機能を持つメソッドと画面の作成

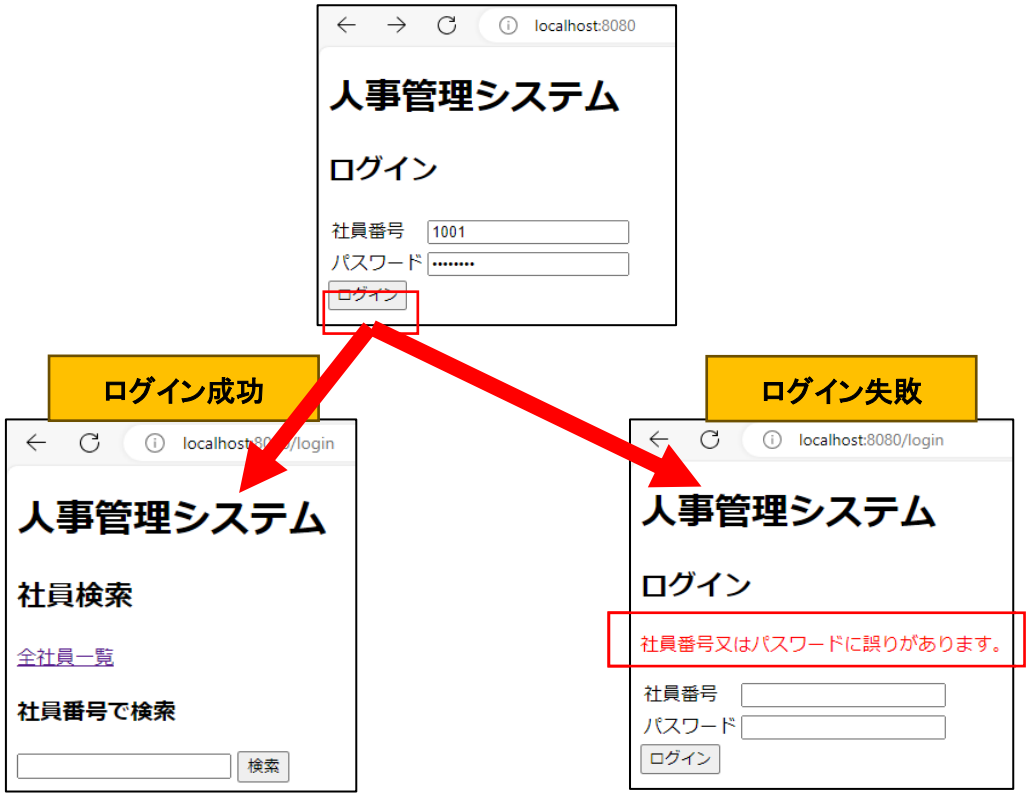
予想所要時間

演習 4.1　10 分

演習 4.2　20 分

演習 4.3　20 分

演習4.1. ID とパスワードからのログイン機能の実装



■ **社員番号とパスワードを入力してもらうログインフォームを作成します。**

- 入力値に誤りがない場合は検索画面へ遷移
- 入力値に誤りがある場合は、ログイン失敗画面へ遷移し、再入力を促します。

a. src/main/resources/templates/index.html のソースコードを確認してください。

- テキスト 4.1 の【 作業手順②】の内容が該当します。
- name 属性の値を覚えてください。
- form タグの action 属性値を確認してください。

b. EmpRepository に、id と password の AND 検索を行う抽象メソッドを追加してください。

- テキスト 4.1 の【 作業手順③】の内容を参考にしてください。
- 主キーとの AND 検索ですので、必ず 1 行のデータを取得します。よって、戻り値型はエンティティの Employee です。
- この演習では、「 empId」ではなく、「 Id」です。間違えるとエラーが出ます。注意

c. LoginController の login メソッドの引数を入れてください。

- テキスト 4.1 の【 作業手順④】を参考にしてください。
- a.で送信したデータを受け取るメソッドです。
  (POST で送信しているので、@PostMapping です)
- login メソッドの引数は、id と password の変数で受け取っても良いですし、empForm を使って受け取っても大丈夫です。

d. ID とパスワードの AND 検索を行い、ユーザデータを取得してください。

- b.で追加したメソッドを呼び出します。
- 引数には、入力された id と password を入れてください。

e. 取得したデータが null なら、“login_fail.html”へ遷移させる条件文を書いてください。

- redirect を使わず、普通に login_fail.html へ return するようにしてください。
- null 以外なら、正しい入力ということなので、演習 2 で実装した「 検索画面」の “emp/index.html”へ遷移させます。

f. src/main/resources/templates/login_fail.html のソースコードを確認してください。

- 既存のログインページ(index.html)の<form>要素と同じコードが書かれていることを確認してください。action 属性が同じですので、同じメソッドを呼び出します。
- 「 社員番号又はパスワードに誤りがあります。」が記入されていることを確認してください。

■ **動作確認：http://localhost:8080/**

- **＜正しいアカウント情報入力＞**

  社員番号：1001
  パスワード：password　を入力し、[ログイン]ボタンをクリックしてください。

  → 正常に検索画面へ遷移できたら、OK です。

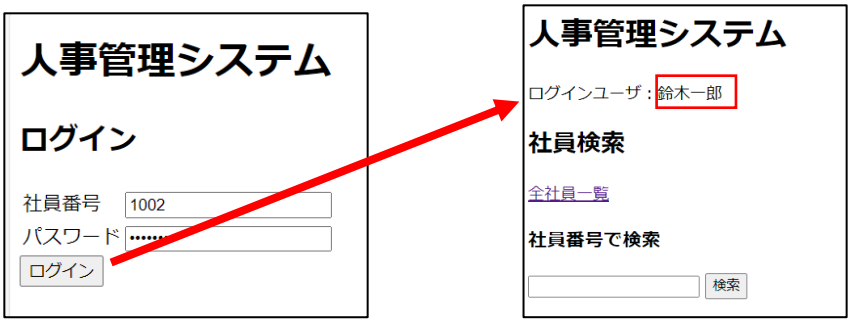
- **＜正しくないアカウント情報入力＞**

  社員番号：1001
  パスワード：pass　を入力し、[ログイン]ボタンをクリックしてください。
  → ログインに失敗し、再入力を促すメッセージが表示されたら、OK です。

演習4.2. セッションデータの取得と利用

■ 正しい ID とパスワードが入力されたことが確認できたら、システム全域で使いまわしできるように、ログインユーザのデータをセッションに格納しておきましょう。

■ **演習 4.1 のソースコードに、追記してください。**



a. LoginController の login メソッドの引数に、セッションオブジェクトを追記してください。

● テキスト 4.2 の【 作業手順①】を参考にしてください。

b. id とパスワードの And 検索から取得したユーザデータをセッションに格納してください。

● 正しい ID とパスワードを入力したら、実行される箇所に追記してください。

● セッションに格納する際のキー名は、「 “user”」にしてください。

c. /resources/templates/emp/index.html にログインユーザ名を表示してください。

● テキスト 4.2 の【 作業手順②】を参考にしてください。

● </hr>の前のに th:text 属性を入れてください。

● セッションに格納していた、“user”を使ってください。

● ログイン画面の index.html ではなく、/emp/index.html ですので、注意しましょう。

d. c.と同じコードを、emp フォルダ内の全ての html ファイルに追記してください。

■ **動作確認：http://localhost:8080/**

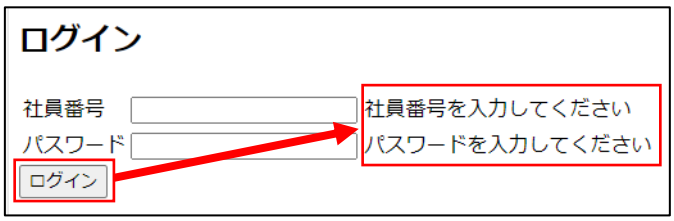
● 社員番号：1001
パスワード：password　を入力し、[ログイン]ボタンをクリックしてください。

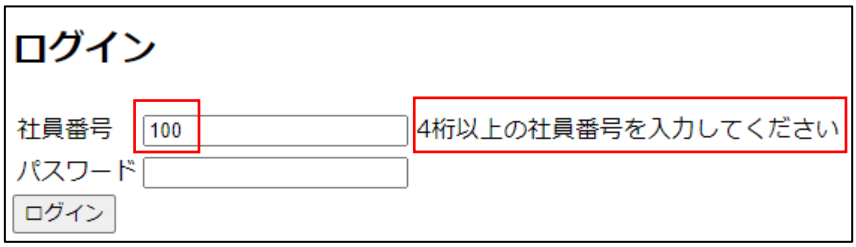
● 社員検索画面に、ログインユーザ名が表示されるかを確認してください。

● 画面を遷移しても、ユーザ名が表示されていることを確認してください。

演習4.3. 入力チェック機能の実装

■ 演習 4.1 では、入力に誤りがある際には、予め用意した login_fail.html へ遷移させる方法を使いました。今度は、index.html 内に入力チェック機能を実装することで、より柔軟で保守性を高めるプログラミングに挑戦しましょう。





a. EmpForm に、以下のような入力チェックができるように、アノテーションを追記してください。

対象	チェック内容	出力メッセージ
社員番号	①未入力時 ②4 桁未満の社員番号入力時	社員番号を入力してください 4 桁以上の社員番号を入力してください
パスワード	①未入力時	パスワードを入力してください

- テキスト 4.3 の【 作業手順①】を参考にしてください。
- 「 未満」については、「 4.3.1 その他の validation アノテーション」の@Min を参考にしてください。(文字ではなく、数字の文字数チェックはなく、数値の大きさで判定)
- @Min(value = 1000, message = “”)のように使います。
- 複数のオプションは、,(カンマ) で並べることができます。

b. LoginController の showLoginView メソッドの引数に、Form クラスを入れてください。

- テキスト 4.3 の【 作業手順②】を参考にしてください。
- この手順を書き忘れると、エラーが出ます。

c. LoginController の login メソッドの引数の部分を、入力チェックができるように修正してください。

- テキスト 4.3 の【 作業手順③】を参考にしてください。
- セッション関連は、そのまま残してください。

d. 入力にエラーがある場合は、ログイン画面へ戻すように条件文を入れてください。

- And 検索を行うコードの前に、記入してください。
- 入力エラーがある場合は、“index”へ return してください。
- And 検索を行う時の login_fail.html へ遷移する条件文は、残しても大丈夫です。

e. /resources/templates/index.html のコードを修正してください。

- テキスト 4.3 の【 作業手順④】を参考にしてください。
- form の属性に、th:object を追記してください。login メソッドの form の変数名と一致させる必要があります。
- テキストフォームの name 属性の代わりに、th:field 属性に修正してください。
- エラーがある場合、メッセージを出力できるように、<td></td>の間にを入れて、th:errors 属性を付与してください。

■ **動作確認：http://localhost:8080/**

- **＜正しいアカウント情報入力＞**

  社員番号：1001
  パスワード：password　を入力し、[ログイン]ボタンをクリックしてください。
  → 正常に検索画面へ遷移でき、ユーザ名が表示されたら、OK です。

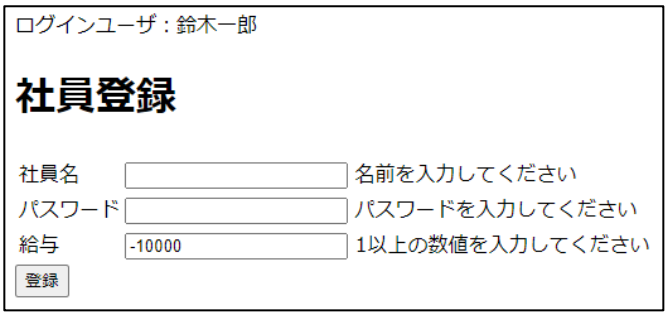
- **＜何も入力しない＞**

  社員番号：【 空欄】
  パスワード：【 空欄】　の状態で [ログイン]ボタンをクリックしてください。
  → ログインに失敗し、再入力を促すメッセージが表示されたら、OK です。

- **<社員番号を 4 桁未満入力>**

  社員番号：100
  パスワード：任意　の状態で [ログイン]ボタンをクリックしてください。
  → ログインに失敗し、桁数についてのメッセージが表示されたら、OK です。

演習4.4.【オプション】登録画面に入力チェック機能の実装



ログイン画面以外にも、登録画面にも入力チェックは必要ですので、以下のような仕様で入力チェック機能を実装してください。社員番号のアノテーションは削除してください。

対象	チェック内容	出力メッセージ
~~社員番号（アノテーション削除）~~	~~①未入力時~~ ~~②4 桁未満の社員番号入力時~~	~~社員番号を入力してください~~ ~~4 桁以上の社員番号を入力してください~~
パスワード	①未入力時	パスワードを入力してください
社員名	①未入力時	名前を入力してください
給与	①未入力時 ②入力値が 1 未満の場合	給与を入力してください 1 以上の数値を入力してください

a. 既存の empForm をコピーし、EmpForm2.java を form パッケージ内に作成します。

- 社員番号に付与していたアノテーションは、削除してください。

b. EmpController の showInputView メソッドを修正してください。

- 引数に入れる Form の名前は、a で追加した Form にしてください。

c. EmpController の insertEmp メソッドを修正してください。

- 引数に入れる Form の名前は、a で追加した Form にしてください。

d. emp/emp_input.html を修正してください。

th:object に指定する Form の変数名は、c で指定した form の変数名に一致させてください。

5

演習 5　Thymeleaf と Web デザイン

演習の目標

- CSS や JavaScript を適用できる
- Thymeleaf の属性を利用し、条件による HTML を生成できる
- レイアウト機能を理解し、実装できる

演習の概要

第 5 章で学習した Thymeleaf の基礎文法を確認し、第 4 章で実装したプログラムのデザインの適用や動的出力を体験します。

- ログインページのデザイン適用
- 共通レイアウトを作成し、全画面に適用
- イメージの動的挿入と、権限による動的表示

予想所要時間

演習 5.1　5 分

演習 5.2　20 分

演習 5.3　20 分

演習5.1. CSS と JavaScript の適用

4 章までのログイン画面に CSS と JavaScript を適用しましょう。

css ファイルと js ファイルは定義済みです。

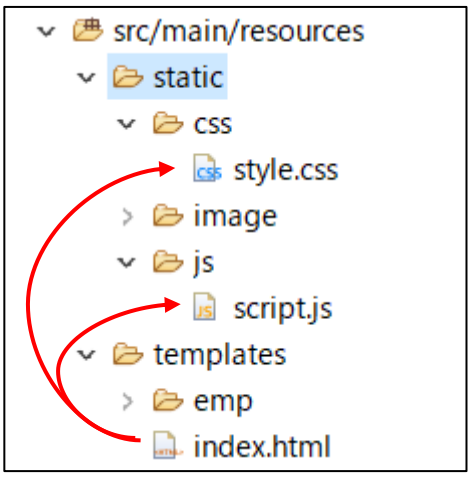
■ ログイン画面(初期画面)



■ ログイン画面(入力エラーがある場合)



a. 以下のファイルのフォルダ構成と、ソースコードを確認してください。



- style.css と、script.js のソースコードの詳細は、コメントを参考にしてください。
- /resources/static の中のファイル構成になっていることを確認してください。

b. /resources/templates/index.html から、上のファイルを読み込んでください。

- テキスト 5.2 を参考にしてください。
- <head>の中に style.css を読み込むコードを書いてください。
- </body>の直前に script.js を読み込むコードを書いてください。

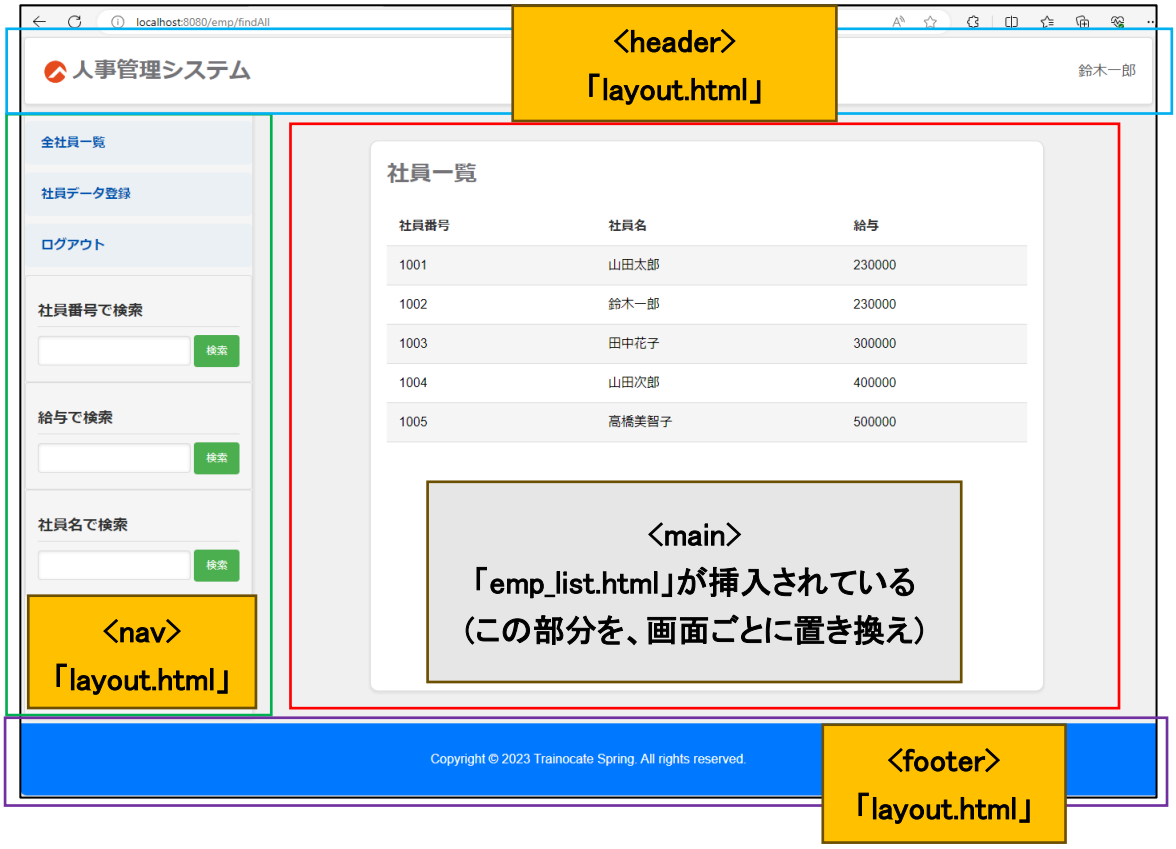
■ **動作確認：http://localhost:8080/**

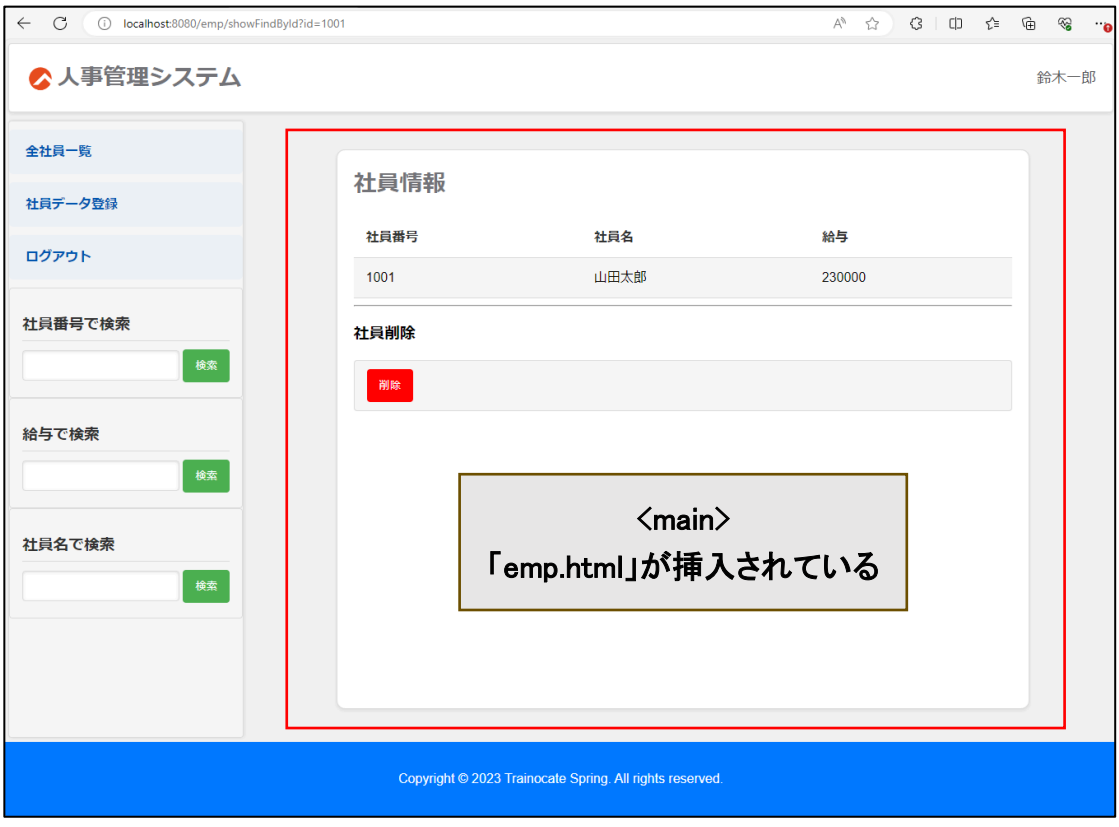
- 新たなデザインが適用されているか確認してください。(css チェック)
- 何も入力せずに、ログインボタンをクリックしてください。(JavaScript チェック)

  → エラーメッセージが点滅しながら回転するのか確認してください。

演習5.2. 共通レイアウトを作成し、全画面に適用

■ 共通レイアウトを作成し、<main>の部分にコンテンツ html を挿入しましょう。





■ 4 章までのソースコードからの変更点

- /emp/index.html が削除され、layout.html に統合されました。
- ログイン成功時の遷移先は、findAll(全社員一覧)になりました。
- 実装済みの「 layout.css」と、「 main_style.css」が追加され、新しいデザインが適用されました。
- サイドバーの存在により、「 メニューに戻る」が削除され、ログインページに戻る「 ログアウト」リンクが追加されました。

a. pom.xml の<dependency>リストに「 thymeleaf-layout-dialect」ライブラリが記載されているか、確認してください。

- テキスト 5.3 の「【 事前準備】 ライブラリを導入」を参考にしてください。

b. /resources/templates/layout.html の<html>をレイアウト機能が使えるように修正してください。

- テキスト 5.3 の【 作業手順①】を参考にしてください。
- <html>の属性に thymeleaf-layout-dialect ライブラリを適用してください。

c. /css/layout.css を適用させるソースコードと、css ファイルも確認してください。

d. <header><nav><main><footer>の構成になっていることを確認してください。

e. <main>の属性に、fragment 属性を追記してください。

- 属性値は、“main_content”にしてください。

f. templates/emp フォルダの中の全ての html ファイルの<html>の属性を修正してください。

- テキスト 5.3 の【 作業手順②】を参考にしてください。
- layout:decorate の属性値は、~{/layout}にしてください。

※コンテンツになる html のデザインは、/css/main_style.css で担当しています。

g. templates/emp フォルダの中の全ての html ファイルの<body>以下のコードを<main>要素で囲って、layout:fragment 属性を付与してください。

- テキスト 5.3 の【 作業手順②】を参考にしてください。
- 属性値は、演習 e.で指定した属性値と一致させてください。

■ **動作確認：http://localhost:8080/**

- 正しい社員情報を入力し、ログインしてください。

  社員番号：1001

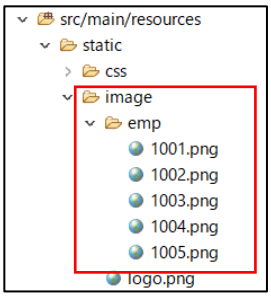
  パスワード：password

- 正常にヘッダー・サイドバー・フッターと、社員一覧が表示されるのか確認してください。
- サイドバーから、他の機能も正常に表示されるかテストしてください。

演習5.3. イメージの動的挿入

■ 社員番号に紐づいているイメージを挿入しましょう。





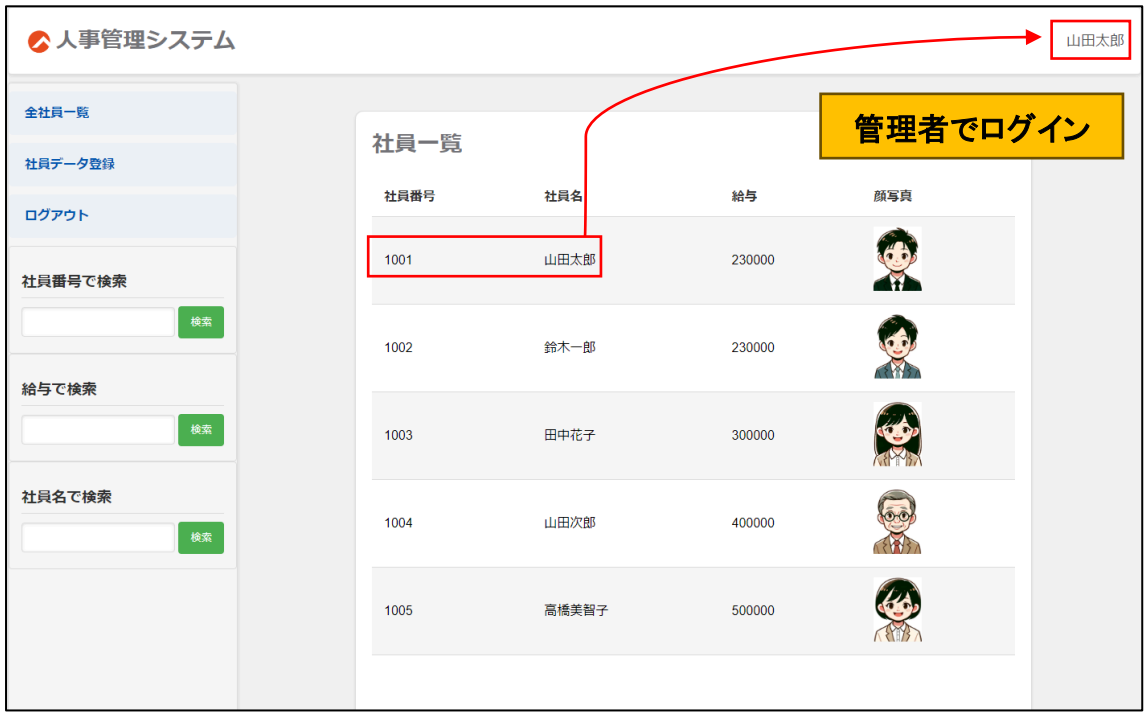
※イメージのファイル名は、社員番号に対応しています。

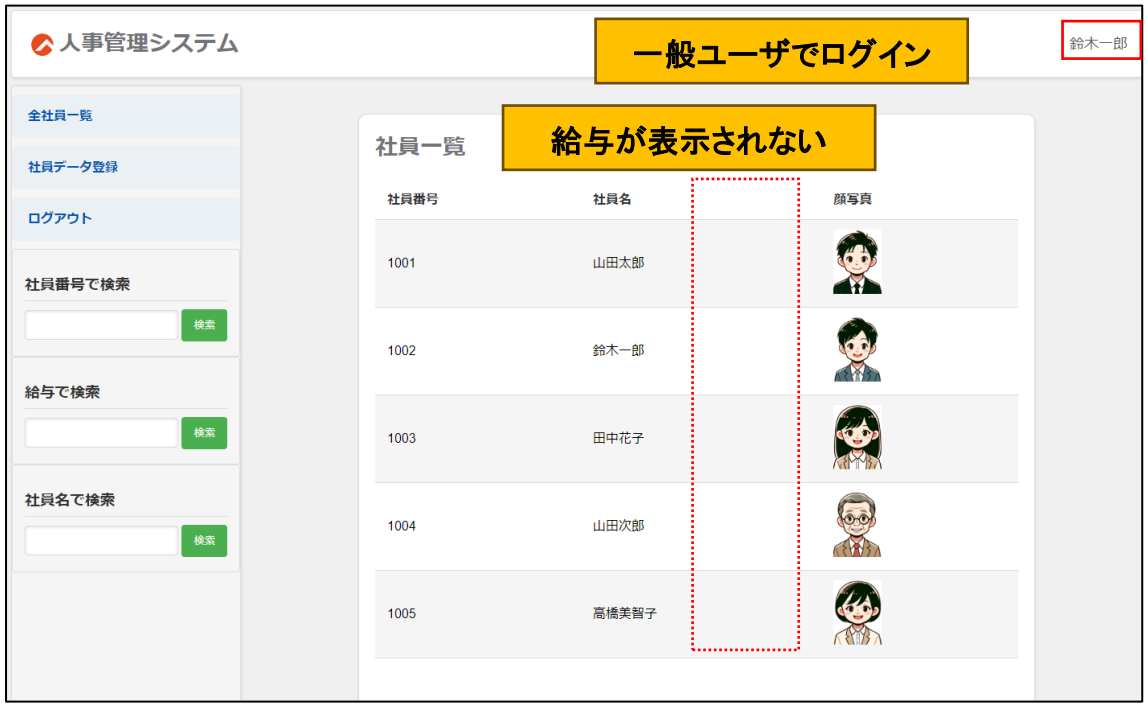
a. /resources/templates/emp/emp_list.html のテーブル内に顔写真の列を追加し、社員番号に該当するイメージを出力してください。

- テキスト 5.1.1 の【 イメージの動的挿入】を参考にしてください。
- テーブルの<th>に、「 顔写真」列を追加してください。
- /image/emp/　の後ろに、「 emp.id」と「 .png」を繋げてください。
- 横幅 60px、高さ 80px のイメージを挿入してください。

演習5.4.【 オプション】 権限による給与データの動的表示

■ 管理者(社員番号 1001)権限のアカウントでログインしないと、給与データをできないように動的表示機能を実装させましょう。





a. emp_list.html の給与列は、ログインユーザの社員番号が 1001 の場合のみ表示するように修正してください。

- テキスト 5.1.2 と、「 ②その他の条件式」の「 (2) 管理者権限に基づく機能の表示」を参考にしてください。
- ログインユーザの特定は、ログイン時に格納した、セッションデータを利用すると便利です。LoginController の login メソッドを確認し、session のキーが「 user」だということを考慮してください。
- 給与列の<th>の部分に th:if 属性を追加してください。
- 給与列のデータ出力の部分も th:if 属性を追加してください。

演習5.5.【 オプション】顔写真の全画面適用

- 演習 5.3 では、emp_list.html のみ顔写真を挿入しましたが、emp.html にも同じ機能を追加してください。

演習5.6.【 オプション】権限による「 給与検索」の動的表示

- 管理者でログインしたユーザのみ、「 給与検索」を表示する実装をしてください。

6

演習 6　外部参照と複雑な SQL 操作

演習の目標

- 外部参照を持つエンティティを定義し、テーブル間のリレーション（1 対多、多対 1）を正しく設定できる
- Spring Data JPA を用いて、外部参照先のデータを含む検索・登録処理を実装できる
- JPQL を使って柔軟な検索条件を指定し、実務に近い形の複雑な検索を行える

演習の概要

この章では、第 6 章で学習した「外部参照のあるエンティティの構成方法」や「JPQL による実践的な検索処理」を確認します。

- location テーブルを新たに作成し、employee テーブルと外部参照で紐付ける
- Employee エンティティと Location エンティティのリレーションを構築
- 外部参照を持つ状態での全件検索・主キー検索・登録処理の実装
- JPQL を用いた条件検索（給与と勤務地）を実装

予想所要時間

演習 6.1　外部参照を定義したエンティティの修正：10 分

演習 6.2　全件検索で外部参照の内容を表示：10 分

演習 6.3　主キー検索で外部参照の内容を表示：10 分

演習 6.4　外部参照付きの登録処理の実装：15 分

演習 6.5　登録画面の作成(セレクトメニュー) ：15 分

演習 6.6　JPQL による条件検索(給与＋勤務地) ：20 分

■追加データベース情報

※データベースは、既に構築済みです。

src/resources/application.properties を確認してください。

＜接続情報＞

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

＜employee テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	−	−	−
3	社員名	name	VARCHAR	−	−	−

4	給与	salary	INT	−	−	
5	勤務地番号	location_id	INT		FOREIGN KEY	外部キー

<Employee テーブルの初期データー>

ID	password	name	salary	location_id
1001	password	山田太郎	230000	1
1002	password	鈴木一郎	230000	2
1003	password	田中花子	300000	3
1004	password	山田次郎	400000	1
1005	password	高橋美智子	500000	1
1006	password	平恵	270000	5
1007	password	佐々木舞	330000	4
1008	password	中村翔	350000	3

＜location テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	勤務地番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	勤務地名	name	VARCHAR	−	−	−

< location テーブルの初期データ>

id	name
1	tokyo
2	osaka
3	fukuoka
4	sapporo
5	okinawa

演習6.1. エンティティ・リポジトリの追加と修正

a. Location エンティティを確認してください。

- jp.co.trainocate.enshu.entity パッケージに、Location.java はすでに作成済みです。
- 以下の内容を確認してください。
    - クラスに@Entity、@Table アノテーションを付けること
    - id フィールドを主キー(@Id)に設定し、自動採番(@GeneratedValue)にすること

【ヒント】

- テキスト 6.1 の【 作業手順②】を参考にしてください。
- 参照関係の定義は、後ほど行いますので、不要です。

b. Employee エンティティを修正してください。

- jp.co.trainocate.enshu.entity パッケージに、Employee.java はすでに作成済みです。
- 以下の内容を追記してください。
    - Location 型の location フィールドを追加すること
    - @ManyToOne と @JoinColumn を使い、location_id カラムと外部参照の関連付けを行うこと
        - @JoinColumn(name = "location_id")と指定してください。

【ヒント】

- テキスト 6.1 の【 作業手順③】を参考にしてください。

c. Location エンティティに参照元(Employee)を持たせましょう。

- jp.co.trainocate.enshu.entity パッケージの Location.java を修正してください。
- 以下の内容を追記してください。
    - Employee エンティティを格納するリストフィールドを作成すること
    - @OneToMany(mappedBy = "location") アノテーションを付けること
        - mappedBy には、Employee エンティティ側で定義したフィールド名("location")を指定してください。

- フィールド名は、employeeList としてください。

【ヒント】

- テキスト 6.1 の【 作業手順②】を参考にしてください。
- 「1 つの勤務地に複数の社員が所属する」関係を表現します。
- リスト型は java.util.List をインポートしてください。

d. Location エンティティ用の Repository インタフェースを確認してください。

- jp.co.trainocate.enshu.repository パッケージ内に、LocationRepository.java が作成されています。確認してください。
- Location エンティティを対象とした Repository です。

【ヒント】

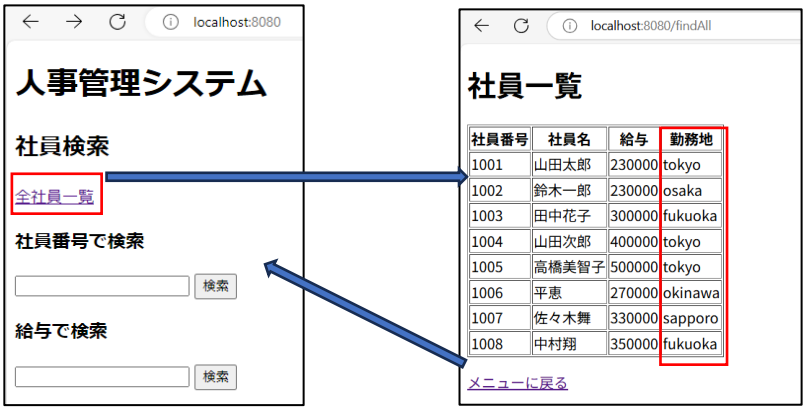
- JpaRepository を継承することで、findAll(), findById(), save() などが自動的に使えるようになります。

演習6.2. 外部参照を持つデータの全件検索

■Employee と Location のリレーションを利用して、全件検索(findAll)を行います。

■取得したリストデータを、一覧表示ページに出力できるようにします。

■Controller 側と HTML 側で、役割に応じた修正を行います。



a. Controller クラスの全件検索処理(findAll メソッド)を完成してください。

- jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
- Repository を使って全社員データを取得し、Model に格納してください。
- Model には、リストを"empList"という名前で格納してください。
- 遷移先の HTML ファイルは、emp_list.html としてください。

【ヒント】

- Repository の findAll()メソッドを利用します。
- location の name も一覧で表示できるようにデータを準備します。

b. employee_list.html に一覧出力する処理を追加してください。

- すでに用意されている emp_list.html ファイルを修正してください。
- empList リストから、社員番号、社員名、給与、**勤務地名**を表示してください。
- Thymeleaf の th:each 構文を利用してください。

【ヒント】

- テキスト 6.2 の【 作業手順②】を参考にしてください。

動作確認：http://localhost:8080/
「 全社員一覧」リンクをクリックし、社員リストが表示されるか確認してください。

演習6.3. 社員番号を使った個別検索

■社員番号(id) で 1 件検索できる機能を作ります。

■検索結果を画面に正しく出力できるようにします。

a. Controller クラスの主キー検索処理(findById メソッド)を完成してください。

- jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
- Repository を使って、指定された id の社員データを取得してください。
- 取得した Employee オブジェクトを"emp"という名前で Model に格納してください。
- 遷移先は emp.html にしてください。

【ヒント】

- URL マッピングは /showFindById としてください(3 章と同じ形式)。

b. emp.html に 1 件分のデータを出力するコードを追加してください。

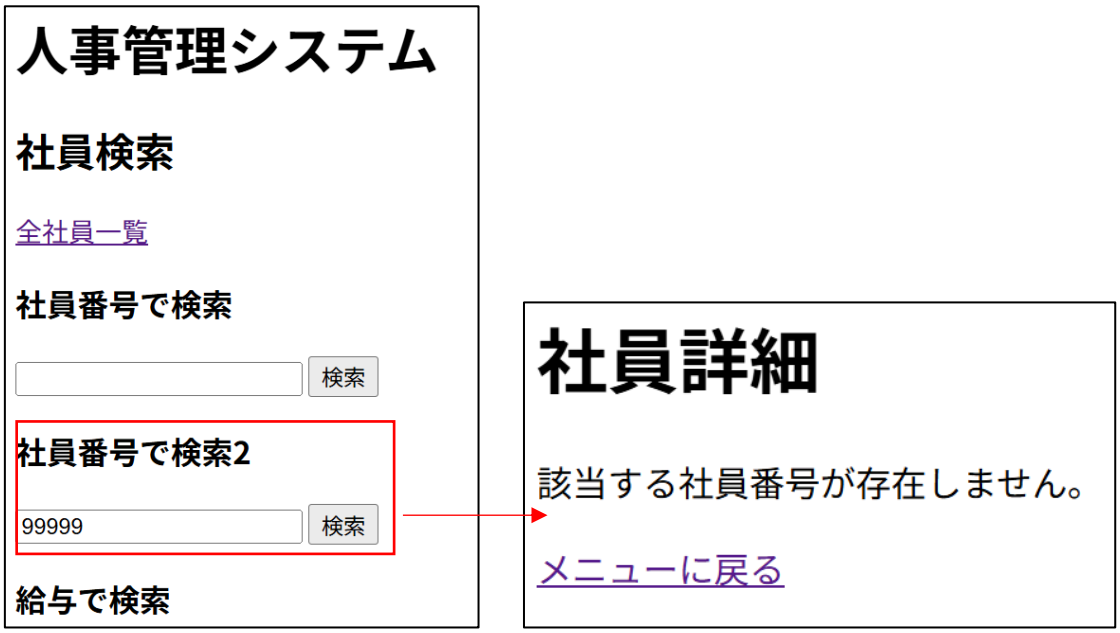
- すでに用意されている emp.html ファイルを修正してください。
- 社員番号、社員名、給与、**勤務地名**を表示するようにしてください。

【ヒント】

- データ 1 件だけなので、th:each は不要です。

動作確認：http://localhost:8080/
「 社員番号で検索」に、「 1001」を入力し、検索します。社員が表示されるか確認してください。

演習6.4.【 オプション】存在しない社員番号を考慮する個別検索



■findById(id).orElse(null) を使って、存在しない値でも安全な主キー検索を実装します。

■社員が見つからない場合、エラーメッセージを表示する処理を実装します。

a. Controller クラスに存在しない値でも安全な主キー検索メソッドを完成してください。

- jp.co.trainocate.enshu.controller.EmpController に新しくメソッドを追加してください。
- Repository の findById(id).orElse(null) を使って検索してください。
- 取得できた Employee オブジェクトを ″emp″ という名前で Model に格納してください。
- 遷移先は **emp2.html** にしてください。

【 ヒント】

- if 文などで null 判定は不要です。orElse(null)により、null かオブジェクトかが自動で取得できます。
- テキスト 6.2.1 の【 作業手順①】を参考にしてください。

b. emp.html でデータ存在チェックを追加してください。

- すでにある **emp2.html** ファイルに、表示切り替えの処理を追記してください。
- 社員データが存在しない場合は、「該当する社員番号は存在しません。」と表示してください。
- th:if と th:unless を使いましょう。

【ヒント】

- th:if="${emp == null}" と th:unless="${emp == null}" を使い、切り替えます。
- テキスト 6.2.1 の【作業手順②】を参考にしてください。

動作確認：http://localhost:8080/

「社員番号で検索 2」に、「9999」を入力し、検索します。

「該当する社員情報は存在しません。」が表示されることを確認します。

演習6.5. 社員情報の登録



■問題構成

小問	内容
a	入力画面へ遷移する Controller メソッドの作成
b	location テーブルのリストを使って、<select> メニューを表示
c	EmpForm に locationId を追加
d	登録処理の実装(save 処理)
e	外部キーの登録時に必要なエンティティ側の設定を確認

a. 入力画面に遷移するためのメソッドを EmpController に追加してください。

- 遷移先の HTML ファイルは emp_input.html にしてください。
- URL マッピングは /showInputView にしてください。
- 画面に <select> を表示するために、Location の全件を取得し、"locations" という名前で Model に格納してください。

【ヒント】

- LocationRepository から findAll() を使って、Location のリストを取得できます

b. emp_input.html に、location を選択できるセレクトボックスを作成してください。

- Form 内で、社員名、給与、パスワード、勤務地(location)を入力できるようにしてください。
- Location の選択肢は、locations というリストを th:each でループさせて出力してください。
- 選択された勤務地は、**name="locationId"** でサーバーへ送信してください。

【ヒント】

- テキスト 6.5 の【 作業手順①】を参考にしてください。

動作確認：http://localhost:8080/
「 社員データ登録」リンクをクリックします。勤務地の選択肢が正しく表示されるのかを確認します。

c. EmpForm に locationId フィールドを追加してください。

- 登録画面で選択された勤務地(locationId)を受け取るために、EmpForm にフィールドを追加してください。
- データ型は Integer としてください。
- name 属性は "locationId" ですので、フォームのフィールド名と一致させてください。
- @Data アノテーションがすでに付いているため、getter/setter は不要です。

【ヒント】

- select タグの name="locationId" に合わせたフィールド名にしてください。
- salary と同じように、Integer 型で定義すれば OK です。

d. EmpController に登録処理メソッド insertEmp を実装してください。

- EmpForm から受け取った値を使って、Employee エンティティを生成してください。
- Employee エンティティには Location location という参照フィールドがあるため、locationId の値をそのまま setLocation() することはできません。
- そのため、**Employee エンティティを開いて、 locationId を別フィールドとして追加し、登録時はそちらを利用してください。**
- リポジトリの save() メソッドで DB に登録してください。
- 登録後は、 /findAll にリダイレクトしてください。

【ヒント】

- Employee エンティティ内の location フィールドは Location 型です。一方、Form からは Integer 型の locationId が送られてくるため、直接代入はできません。
  代わりに locationId を登録用に持たせておき、 location フィールドには insertable = false, updatable = false を指定しておく必要があります。
- **テキスト 6.5 の【 作業手順④・⑤】を参考にしてください。**

フィールド	説明
location(Location 型)	外部テーブルと結びつけるためのオブジェクト。SELECT では自動で JOIN してくれるが、登録・更新には使わない。
locationId(Integer 型)	フォームから渡された実際の整数値を使って登録・更新を行うためのフィールド。DB のカラムに直接反映される。
insertable = false, updatable = false	location(参照オブジェクト)を登録や更新に使わないように Hibernate に明示するための設定。

動作確認:http://localhost:8080/

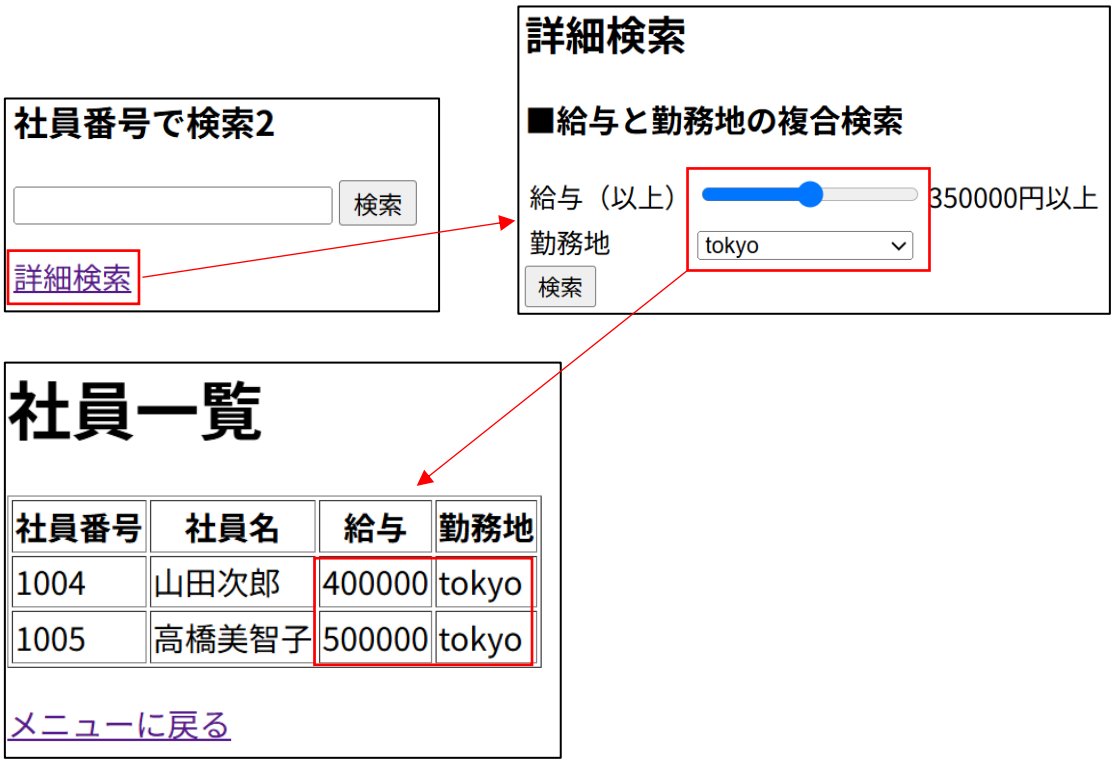
「 社員データ登録」リンクをクリックし、任意のデータを入れ、登録します。正しく登録されるのかを確認してください。

演習6.6. JPQL を使った検索(給与+勤務地)

給与と勤務地の AND 検索は、SQL 文にすると以下のようになります。

「 SELECT *
FROM employee e JOIN location l
ON e.location_id = l.id
WHERE e.salary >= ? AND l.name = ?」

複雑な検索条件を JPQL で柔軟な検索条件を記述し、給与と勤務地に一致する社員を検索できるようにしましょう。



a. EmpController の showSearchForm メソッドのソースコードを確認してください。

1．演習 6.5 と同様に、検索画面の表示のためにも勤務地リストの取得が必要です。

b. 入力画面(emp_search.html)が、以下の内容で作成されていることを確認してください。

1. 「 /showSearchForm」でアクセスすると検索入力画面が表示される
2. 入力画面には以下の構成がある:

入力項目	内容
給与	type=″range″ で、給与を 200,000～500,000 の範囲から選べるようにしている
勤務地	DB の Location テーブルから name を取得し、セレクトメニューに表示している

【ヒント】

- range を使うとスライダー式で給与を選べます。
- th:each="loc : ${locationList}" を使って、セレクトメニューに DB の勤務地一覧を表示できます。
- 今いくらに設定されているのかの確認のため、一部 JavaScript のコードが混ざっています。

c. EmpRepository に、給与と勤務地を条件にした JPQL を使った検索メソッドを実装してください。

- 戻り値は List<Employee> としてください。
- メソッド名は findHighSalaryByLocation としてください。
- @Query アノテーションを使って、JPQL を記述してください。
- 引数は int salary、String locationName の 2 つを受け取れるようにしてください。

【ヒント】

- エンティティ名は Employee(JPQL では、テーブル名ではなくエンティティ名を使います。)
- location テーブルは @ManyToOne で結ばれているため、e.location.name のように書くことができます。

- WHERE 句には e.salary >= :salary AND e.location.name = :locationName のように書きます。
- テキストの 6.4.2 の部分を参考にしてください。

d. EmpController に、以下の条件で JPQL による検索を実行するメソッドを追加してください。

内容	詳細
URL	/findHighSalary
HTTP メソッド	GET
引数	int salary, String locationName, Model model
使用するリポジトリメソッド	findHighSalaryByLocation()(演習 6.6.c で実装済み)
HTML ファイル	検索結果は emp_list.html に表示
Model への追加	取得したリストを "empList" のキー名で格納

動作確認：http://localhost:8080/

「 詳細検索」リンクをクリックし、給与を「 400000 円」以上、勤務地は「 tokyo」に設定し、検索してください。2 件のデータが表示されるかを確認してください。

7

演習 7　Bootstrap

演習の目標

- Bootstrap の導入方法(CSS・JS の読み込み)を理解し、導入前後の画面の変化を体感できる
- Bootstrap の基本クラス(色・テキスト・配置・ボタン・枠線・余白・フォーム)を適切に使い、Web ページの外観を整えられる
- グリッドシステムを使って、レイアウト(ヘッダー・サイドバー・メイン画面)の構成を柔軟に設計できる
- 共通レイアウト機能(テンプレート化)を使って、コードの再利用性・保守性を向上できる

演習の概要

この章では、第 7 章で学習した Bootstrap の基礎知識を踏まえ、実際にシステム画面のスタイルを改善します。

- Bootstrap の CSS・JS ファイルを読み込んだ状態と、読み込まない状態を比較
- Bootstrap のスタイルクラス(色・文字サイズ・ボタン・配置など)をフォームやテーブルに適用

- グリッドシステムを活用し、メニューとメインコンテンツを左右に分けて表示
- 共通レイアウト(layout.html)を作成し、ヘッダーやサイドバーをすべての画面で共通化
-

予想所要時間

演習 7.1　Bootstrap 導入前後の変化を確認:5 分

演習 7.2　Bootstrap の基本スタイルの適用:15 分

演習 7.3　Bootstrap のテーブルデザインの適用:10 分

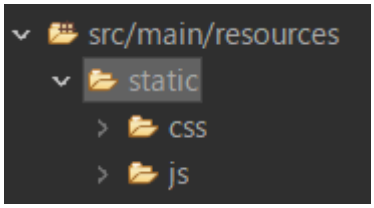
演習 7.4　グリッドシステムで画面レイアウトを整える:15 分

演習 7.5　フォームの改善:10 分

演習 7.6　演習 7.6.【 オプション】 共通レイアウト画面に変更:20 分

■事前準備(※操作不要)

次のフォルダに Bootstrap の CSS / JS ファイルが事前に格納されています。
特にファイルをダウンロードしたり、CDN を使って読み込んだりする必要はありません。



ファイル名	格納場所
bootstrap.min.css	src/main/resources/static/css/
bootstrap.bundle.min.js	src/main/resources/static/js/

演習7.1. Bootstrap 導入前後の変化を確認



- Bootstrap の CSS と JS ファイルを読み込むことで、Web ページの見た目や動作がどう変わるかを体感しましょう。

- 特に、ドロップダウンメニューの動作に注目してください。

a. index.html に記載されているソースコードを確認してください。

- CSS と JS の読み込み部分がコメントアウトされています。

b. そのままブラウザで「 localhost:8080」 にアクセスして、ページの見た目と動作を確認してください。

- ページは正常に表示されますか？
- 「 メニュー」ボタンを押しても反応がありますか？

c. コメントアウトされていた css と js の 2 か所のコメントを解除して、再度ブラウザで表示を確認してください。

d. 再読み込み後、「 メニュー」ボタンを押して、表示に変化があるか確認してください。

ヒント

- Bootstrap は、class 属性だけでデザインが適用される仕組みですが、そのためには CSS の読み込みが必須です。
- また、メニューの展開やモーダルの表示など、動きのある要素には JavaScript の読み込みも必要です。
- JS が読み込まれていないと、ドロップダウンボタンを押しても何も起きません。

演習7.2. Bootstrap の基本スタイルの適用



■ **ブラウザから、「 localhost:8080/2」を開いて、修正していく演習です**

■ **index2.html に Bootstrap のクラスを適用して、画面全体の見た目を整えてください。**

以下の指示に従って、既存の HTML にクラスを追加・修正してください。

(※クラスの追加のみで構いません。HTML の構造は変えなくて OK です)

■ **適用されたことが確認できたら、テキストを参照し、class を変え、変化を確認してください。**

ヒント

- 色に関するクラスは .text-*(文字色)と .bg-*(背景色)を使い分けましょう。
- ボタンには btn と btn-* をセットで使います。
- .m-(margin)、.p-(padding)は -start, -end, -top, -bottom など方向を指定できます。

a. 全体の背景と本文余白の追加

```
<!-- body タグを修正してください -->
<body class="bg-light p-4">
```

- bg-light : 全体を淡い灰色の背景色に
- p-4 : ページ内に十分な余白を追加

b. タイトル部分に装飾を加えてください

```
<h1 class="text-center text-primary fw-bold mb-4">人事管理システム
</h1>
```

- 中央揃え ＋ 青色 ＋ 太字 ＋ 下に余白

c. 各セクションタイトルの見た目を整えてください

```
<h2 class="text-success border-bottom pb-2 mb-4">社員検索</h2>
```

- 緑色 ＋ 下線風のボーダー ＋ 下に余白

d. フォームの検索ボタンを以下のように修正してください

```
<input type="submit" value="検索" class="btn btn-primary ms-2">
```

- btn と btn-primary でデザイン済みボタンに
- ms-2 で左側に少し余白(margin-start)

演習7.3. Bootstrap のテーブルデザインの適用



■ **Bootstrap のクラスを使って、社員一覧テーブルを見やすく整える方法を学びます。**

■ **ブラウザから、「 localhost:8080/findAll」を開いて、メニュー修正していく演習です**

a. emp_list.html に記載されている現在のテーブル構造を確認してください。

- 〈table〉 に border="1" が直接指定されています。
- 〈tr〉 と 〈td〉 だけで構成されており、Bootstrap のクラスは一切使われていません。

b. テーブル構造を Bootstrap に合わせて書き換えましょう。

以下のように、HTML の構造を改善してください。

修正ポイント	内容
〈table〉	border="1" を削除し、Bootstrap の .table クラスを追加する
〈thead〉	ヘッダー行(見出し)を囲むタグとして追加する
〈tbody〉	データ行(繰り返し部分)を囲むタグとして追加する

ヒント

- .table クラスは、Bootstrap で最も基本的な装飾クラスです。
- 今回の演習では、まず .table だけを付け、見た目の変化に注目します。

- 補助的に .table-hover や .table-striped を後の演習で追加します。

c. <thead> に .table-primary クラスを追加してください。

- 見出し行(ヘッダー行)が薄青色の背景になります。
- ユーザが表の構造をひと目で理解しやすくなります。

d. <table>に以下の Bootstrap のクラスを適用してください。

クラス名	効果
table	Bootstrap 標準のテーブルデザイン
table-bordered	枠線をつける
table-hover	ホバー時に行の背景色が変わる

e. <tbody>に、.table-light を追加してください。

- テーブルの本文の部分の背景色が、薄いグレーになります。

演習7.4. グリッドシステムで画面レイアウトを整える



社員検索ページを、Bootstrap の「グリッドシステム」を使って、左にサイドメニュー、右にメイン画面を表示する形式に変更していきます。

a. 上記の完成後の画面を見て、どのような分割がされているのか確認してください。

- 「全社員一覧」「詳細検索」「社員データ登録」のボタンが左側の縦方向に並ぶメニューとして表示されています。
- その右側には、社員情報のテーブル一覧が表示されています。
- 上記 2 つのエリアが横に並んでいることを確認しましょう。
- **演習 URL は、localhost:8080/3 です。**

【ヒント】

- Bootstrap のグリッドシステムでは、1 行(row)を 12 のカラム(列)に分割できます。
- 今回は**「左が 3 カラム」「右が 9 カラム」**で構成されているようです。

b. emp_list2.html を開き、 と で画面全体を囲んでください。

- 「画面全体」とは、<body>から、</body>直前までを意味します。
- 「container」クラスは、ページの横幅を制御し、中央に配置させる役割を持ちます。
- 「row」クラスは、グリッドシステムでカラム(列)を並べる土台になります。

c. 左を 、右を に分けてください。

- 「col-md-3」:左側のメニューや検索フォームを入れる枠です(横幅 3/12＝25%)
- 「col-md-9」:右側のメインコンテンツ(社員一覧)を表示する枠です(横幅 9/12＝75%)

【ヒント】

- col-md-* という記法は、「中サイズ以上の画面ではこの比率にする」という意味です。
- md を使えば、スマホ画面では縦並び、PC 画面では横並びのように、画面サイズによって自動で切り替わる「レスポンシブデザイン」が実現できます。

演習7.5. フォームの改善



Bootstrap の form 関連のクラスを適用し、レイアウトを整える演習を行いましょう。

サイドバーの検索フォームの修正と、社員登録フォームを修正します。

a. emp_input.html を開き、サイドバーの検索フォームを確認してください。

現在のコード(修正前)では、<input> と <button> が縦並びで配置されており、見た目に一体感がありません。

b. Bootstrap 式のフォームのデザインを適用し、ボタンを横並びにしてください。

- カラムを、8:2 で分けて配置する方法もありますが、今回は、input を囲っている div 要素に「.input-group」を付与してください。

ヒント

- form-control は入力欄に付ける
- 親要素に input-group を付けることで横並びになる

c. メイン画面の登録フォームのデザインを整えます。<input>や<select>に適切なクラスを付与してください。

- 入力フォームの名称のため、label を使っているコードを確認してください。
- 各入力欄には、Bootstrap のフォーム用クラス form-control を指定します。
- セレクトメニューには、form-select を適用します。

演習7.6.【 オプション】 共通レイアウト画面に変更

5 章のレイアウトの部分を参考し、画面ごとに繰り返し記述されていた **ヘッダーやメニュー（ サイドバー）** を共通化し、保守性と再利用性の高いレイアウト構造を構築しましょう。

a. pom.xml に、レイアウト関連の dependency が導入されているか確認します。

b. templates/layout フォルダの layout.html を開いて、コードを確認してください。

c. ヘッダやサイドバーなどの、共通のコードを貼り付けします。メイン画面の部分は、以下のようにしてください。

```
<div class="col-md-9" layout:fragment="main_content">
        <!-- ここに各画面固有のコンテンツが表示されます -->
</div>
```

d. emp_list3.html の の中には、一覧画面のコードを入れてください。

e. emp_input2.html にも同様に、社員登録のコードを入れてください。

f. その他の画面も、Bootstrap のデザインを適用してください。

■動作確認

http://localhost:8080/4

無断複写・複製の禁止
本書は著作権法上の保護を受けています。
本書の全部または一部を無断で複写複製することは
禁じられています。

Spring Boot 入門
～サーブレット&JSP からのステップアップ～ 演習ガイド

2024 年 02 月06 日　第 1 版　発行
2024 年 03 月15 日　第 2 版　改訂　著　者
2024 年 12 月 04 日　第 3 版　改訂　発行者
2026 年 03 月 13 日　第 4 版　改訂

トレノケート株式会社

〒 163-6027
東京都新宿区西新宿 6 丁目 8 番 1 号
住友不動産新宿オークタワー 27 階
電話 03-3347-9686

https://www.trainocate.co.jp/

落丁本、乱丁本はお取り替えいたします。
© 2024 Trainocate Japan, Ltd.

JAC0177G-L1-04